

1. Search a Target in a Large Dataset

Your explanation is correct, but the table should distinguish **search time** from the **cost of sorting**.

Dataset Size (N) Linear Search Binary Search*

1,000	$O(N)$	$O(\log N)$
10,000	$O(N)$	$O(\log N)$
1,000,000	$O(N)$	$O(\log N)$

*If the data is already sorted. If it isn't, sorting adds **$O(N \log N)$** preprocessing time.

Expected Result: Binary Search is significantly faster for repeated searches on a sorted dataset. For a single search on unsorted data, sorting first may not be worthwhile.

2. Sorting Large Data Efficiently

The main correction is that **Quick Sort is $O(N \log N)$ on average, but $O(N^2)$ in the worst case.**

Algorithm Average Time Worst Case Stable?

Bubble Sort	$O(N^2)$	$O(N^2)$	Yes
Merge Sort	$O(N \log N)$	$O(N \log N)$	Yes
Quick Sort	$O(N \log N)$	$O(N^2)$	Usually No

Expected Result: Bubble Sort becomes impractical as the dataset grows, while Merge Sort and well-implemented Quick Sort are generally much more efficient.

3. String Concatenation Performance

For C#, remove **StringBuffer** because it is a Java class. The appropriate .NET comparison is string vs. StringBuilder.

Technique Typical Complexity

Repeated string += $O(N^2)$ in common repeated-concatenation scenarios

StringBuilder.Append() Approximately $O(N)$ overall

Expected Result: StringBuilder is generally much more efficient when repeatedly building a large string because string is immutable.

One nuance: modern C# compilers can optimize some simple string concatenations, so the exact performance depends on how the concatenation is written.

4. Large File Reading Efficiency

This section needs the biggest conceptual correction.

StreamReader does **not** simply read one character at a time. It is a buffered text reader that handles character decoding.

FileStream works with **bytes**, making it appropriate for raw/binary data.

Class	Best suited for
-------	-----------------

StreamReader	Reading text files
--------------	--------------------

FileStream	Reading/writing raw bytes and binary data
------------	---

Therefore, it isn't generally correct to say that FileStream is automatically faster than StreamReader. Performance depends on the workload, buffering, encoding, and how the data is processed.

Expected Result: Use StreamReader when the application needs text. Use FileStream when working with bytes or binary data.

5. Recursive vs. Iterative Fibonacci

Your conclusion is correct.

Recursive:

```
public static int FibonacciRecursive(int n)
{
    if (n <= 1) return n;

    return FibonacciRecursive(n - 1)
        + FibonacciRecursive(n - 2);
}
```

Iterative:

```
public static int FibonacciIterative(int n)
{
    if (n <= 1) return n;

    int a = 0, b = 1;

    for (int i = 2; i <= n; i++)
    {
        int sum = a + b;
```

```

    a = b;
    b = sum;
}

return b;
}

```

Approach	Time	Extra Space
Naive Recursive	Exponential, commonly stated as $O(2^n)$	$O(N)$
Iterative	$O(N)$	$O(1)$

The recursive version repeatedly calculates the same Fibonacci values, causing exponential growth. The iterative version calculates each value only once.